



Seguridad y Calidad en el Desarrollo (CDY2203)

Informe escrito

Investigación de Herramientas de Cobertura

Nombre de la herramienta	Tipo	Organización/empresa	Características principales	URL Oficial
JaCoCo	Software Libre (EPL)	EclEmma / Marc Hoffmann	Se integra perfectamente con Maven y Gradle. Mide cobertura de instrucciones, líneas y ramas (branches). Genera reportes HTML detallados.	www.jacoco.org
Cobertura	Software Libre (GPL)	Comunidad Open Source	Herramienta clásica para Java. Se basa en jcoverage. Permite calcular el porcentaje de código ejecutado por las pruebas unitarias.	cobertura.github.io
Clover	Comercial (Ahora Open Source)	Atlassian (Open Source por OpenClover)	Proporciona métricas detalladas y análisis de riesgos. Se integra bien con herramientas de CI/CD como Jenkins, que ya has utilizado antes.	openclover.org

Selección y Justificación

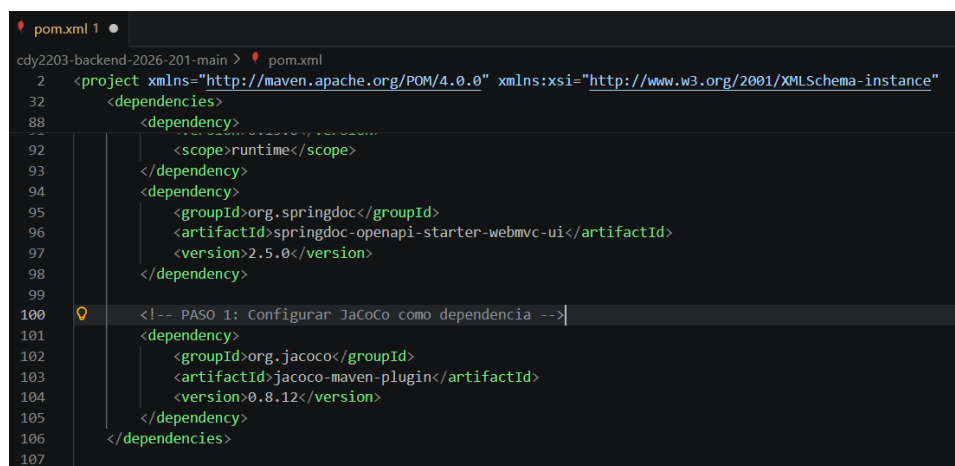
Para este proyecto, la herramienta seleccionada es JaCoCo.

Justificación:

- **Integración Nativa:** Al trabajar con un entorno basado en Maven y Spring Boot, JaCoCo se integra de manera sencilla como un plugin en el archivo pom.xml. Esto evita configuraciones complejas en el sistema operativo Windows y facilita la portabilidad del proyecto.
- **Bajo Impacto en el Rendimiento:** La herramienta utiliza instrumentación de bytecode en tiempo de ejecución, lo que permite recolectar métricas de cobertura de forma eficiente durante la ejecución de las pruebas unitarias sin degradar el rendimiento del proceso de construcción (*build*).
- **Reportes Visuales Detallados:** La capacidad de generar informes automáticos en formato HTML proporciona una auditoría visual clara. Esto permite identificar mediante códigos de colores (líneas verdes y rojas) exactamente qué partes del código han sido validadas, asegurando el cumplimiento del **mínimo del 60% de cobertura** que se pide en el proyecto.

Porcentaje de cobertura de pruebas Unitarias

Agrego la dependencia de Jacoco a la **capa backend** y como dependencia y plugin en el pomp.xml (se debe comentar el plugin de OWASP).



```
cdy2203-backend-2026-201-main > pom.xml
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
32  <dependencies>
88  <dependency>
92  <scope>runtime</scope>
93  </dependency>
94  <dependency>
95  <groupId>org.springdoc</groupId>
96  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
97  <version>2.5.0</version>
98  </dependency>
99
100  <!-- PASO 1: Configurar JaCoCo como dependencia -->
101  <dependency>
102  <groupId>org.jacoco</groupId>
103  <artifactId>jacoco-maven-plugin</artifactId>
104  <version>0.8.12</version>
105  </dependency>
106  </dependencies>
107
```

```

127 <!-- PASO 2: Configurar JaCoCo como plugin -->
128 <plugin>
129   <groupId>org.jacoco</groupId>
130   <artifactId>jacoco-maven-plugin</artifactId>
131   <version>0.8.12</version>
132   <executions>
133     <execution>
134       <id>default-prepare-agent</id>
135       <goals>
136         <goal>prepare-agent</goal>
137       </goals>
138     </execution>
139     <execution>
140       <id>default-report</id>
141       <goals>
142         <goal>report</goal>
143       </goals>
144     </execution>
145     <execution>
146       <id>default-check</id>
147       <goals>
148         <goal>check</goal>
149       </goals>
150       <configuration>
151         <rules>
152           <rule>
153             <element>BUNDLE</element>
154             <limits>
155               <limit>
156                 <counter>COMPLEXITY</counter>
157                 <value>COVEREDRATIO</value>
158                 <minimum>0.60</minimum>
159               </limit>

```

Ejecución de JaCoCo en el proyecto haciendo una limpieza, instalación y reporte completo

```
C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-backend-2026-201-main
(2)\cdy2203-backend-2026-201-main.\mvnw clean install
```

Obteniendo como resultado al ejecutar la app un 57% por eso marca BUILD FAILURE, porque JaCoCo entrega un ejecución exitosa cuando es sobre el 60% que cubren las pruebas unitarias.

```

2026-05-02T19:56:28.972-04:00 TRACE 25824 --- [backend] [ main] m.m.a.RequestMappingHandlerMethodProcessor : Writing ["login-post-ok"]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.786 s -- in com.duoc.backend.WebSecurityConfigTest
[INFO] Results:
[INFO] Tests run: 88, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ backend ---
[INFO] Building jar: C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-backend-2026-201-main (2)\cdy2203-backend-2026-201-main\target\backend-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:4.0.3:repackage (repackage) @ backend ---
[INFO] Replacing main artifact C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-backend-2026-201-main (2)\cdy2203-backend-2026-201-main\target\backend-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-backend-2026-201-main (2)\cdy2203-backend-2026-201-main\target\backend-0.0.1-SNAPSHOT.jar.original
[INFO]
[INFO] --- jacoco:0.8.12:report (default-report) @ backend ---
[INFO] Loading execution data file C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-backend-2026-201-main (2)\cdy2203-backend-2026-201-main\target\jacoco.exec
[INFO] Analyzed bundle 'backend' with 26 classes
[INFO]
[INFO] --- jacoco:0.8.12:check (default-check) @ backend ---
[INFO] Loading execution data file C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-backend-2026-201-main (2)\cdy2203-backend-2026-201-main\target\jacoco.exec
[INFO] Analyzed bundle 'backend' with 26 classes
[WARNING] Rule violated for bundle backend: complexity covered ratio is 0.57, but expected minimum is 0.60
[INFO]
[INFO] BUILD FAILURE
[INFO]
[INFO] Total time: 32.644 s
[INFO] Finished at: 2026-05-02T19:56:31-04:00
[INFO]
[ERROR] Failed to execute goal org.jacoco:jacoco-maven-plugin:0.8.12:check (default-check) on project backend: Coverage checks have not been met. See log for details. -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:

```

Reporte de jaCoCo con un 57% que se logra cumplir de cubrimiento de la app

backend

backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.duoc.backend		67 %		72 %	122	284	193	562	77	182	1	26
Total	641 of 1.984	67 %	57 of 204	72 %	122	284	193	562	77	182	1	26

backend > com.duoc.backend

com.duoc.backend

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
AppointmentController		1 %		0 %	14	15	37	38	7	8	0	1
PetController		71 %		68 %	23	51	16	92	0	8	0	1
Appointment		4 %		n/a	13	14	25	26	13	14	0	1
JWTAuthenticationConfig		5 %		n/a	1	2	16	17	1	2	0	1
Invoice		67 %		66 %	13	23	16	41	11	20	0	1
Patient		39 %		n/a	11	14	16	26	11	14	0	1
UserController		7 %		n/a	1	2	7	8	1	2	0	1
InvoiceLineItem		65 %		50 %	10	19	14	33	8	17	0	1
Pet		70 %		n/a	7	20	13	38	7	20	0	1
UserResponse		0 %		n/a	4	4	8	8	4	4	1	1
MyUserDetailsService		44 %		0 %	2	4	4	7	1	3	0	1
User		68 %		n/a	6	14	6	18	6	14	0	1
LoginRequest		57 %		n/a	3	6	5	11	3	6	0	1
Constants		38 %		n/a	2	3	3	5	2	3	0	1
PasswordMigrationRunner		52 %		50 %	1	4	3	7	0	3	0	1
BackendApplication		37 %		n/a	1	2	2	3	1	2	0	1
SecuredController		50 %		n/a	1	2	1	2	1	2	0	1
InvoiceController		99 %		80 %	8	29	1	60	0	8	0	1
UserService		100 %		100 %	0	26	0	39	0	8	0	1
JWTAuthorizationFilter		100 %		87 %	1	9	0	33	0	5	0	1
PatientController		100 %		100 %	0	5	0	17	0	4	0	1
WebSecurityConfig		100 %		n/a	0	4	0	11	0	4	0	1
LoginController		100 %		100 %	0	3	0	7	0	2	0	1
UserRegistrationRequest		100 %		n/a	0	7	0	10	0	7	0	1
InvoiceLineItemType		100 %		n/a	0	1	0	4	0	1	0	1
OpenAPIConfig		100 %		n/a	0	1	0	1	0	1	0	1
Total	641 of 1.984	67 %	57 of 204	72 %	122	284	193	562	77	182	1	26

Ahora debemos subir ese porcentaje del 57% con la modificación de algunas clases, con el objetivo de que al volver a realizar las pruebas logremos sobre un 60%

Para esto creo la clase AppointmentControllerTest y ModelTests

```
EXPLORER
CDY2203-BACKEND-2026-201-MAIN (2)
  cdy2203-backend-2026-201-main
    src
      main
        java.com.duoc.backend
          userRepository.java
          SecuredController.java
          User.java
          UserController.java
          UserRegistrationRequest.java
          UserRepository.java
          UserResponse.java
          UserService.java
          WebSecurityConfig.java
        resources
          test.java.com.duoc
            backend
              AppointmentControllerTest.java 4
              BackendApplicationTests.java
              InvoiceControllerTest.java
              JWTAuthorizationFilterTest.java
              LoginControllerTest.java
              ModelTests.java 7
              PatientControllerTest.java
              PetControllerTest.java
              UserServiceTest.java
              WebSecurityConfigTest.java
        testsupport

OUTLINE
TIMELINE
LOGICAL STRUCTURE
JAVA PROJECTS
MAVEN

pom.xml 2
AppointmentControllerTest.java 4
ModelTests.java 7

cdy2203-backend-2026-201-main > src > test > java > com > duoc > backend > AppointmentControllerTest.java > AppointmentControllerTest
1 package com.duoc.backend;
2
3 import static org.mockito.ArgumentMatchers.any;
4 import static org.mockito.Mockito.when;
5 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
6 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
7
8 import org.junit.jupiter.api.Test;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
11 import org.springframework.boot.test.context.SpringBootTest;
12 import org.springframework.test.mock.Mockito.MockBean;
13 import org.springframework.http.MediaType;
14 import org.springframework.test.web.servlet.MockMvc;
15
16 // Importa tus entidades y repositorios
17 // Import com.duoc.backend.repository.AppointmentRepository;
18 // Import com.duoc.backend.model.Appointment;
19
20 @SpringBootTest
21 @AutoConfigureMockMvc
22 public class AppointmentControllerTest {
23
24     @Autowired
25     private MockMvc mockMvc;
26
27     @MockBean
28     private AppointmentRepository appointmentRepository;
29
30     @Test
31     public void testCreateAppointment() throws Exception {
32         // Simulamos que el guardado en BD funciona
33         when(appointmentRepository.save(any(Appointment.class))).thenReturn(new Appointment());
34
35         String jsonRequest = "{\"date\": \"2026-05-10\", \"description\": \"Consulta General\"}";
36
37         mockMvc.perform(post("/appointments")) // Ajusta a tu ruta real
```

Creo la clase ModelTests

```
pom.xml 2
AppointmentControllerTest.java 4
ModelTests.java 7

cdy2203-backend-2026-201-main > src > test > java > com > duoc > backend > ModelTests.java > ModelTests
1 package com.duoc.backend;
2
3 import static org.junit.jupiter.api.Assertions.assertEquals;
4 import org.junit.jupiter.api.Test;
5
6 public class ModelTests {
7
8     @Test
9     public void testUserResponse() {
10         UserResponse response = new UserResponse();
11         response.setMessage("Éxito");
12         response.setStatus(200);
13
14         assertEquals("Éxito", response.getMessage());
15         assertEquals(200, response.getStatus());
16     }
17
18     @Test
19     public void testAppointmentEntity() {
20         Appointment appointment = new Appointment();
21         appointment.setId(id: 1L);
22         appointment.setReason(reason: "Control");
23
24         assertEquals(1L, appointment.getId());
25         assertEquals("Control", appointment.getReason());
26     }
27 }
```

Pruebas unitarias en **capa Frontend**

Agrego la dependencia de Jacoco a la capa Frontend como dependencia y plugin en el pomp.xml (se debe comentar el plugin de OWASP).

```
67 <!-- Paso 1: JaCoCo como dependencia -->
68 <dependency>
69   <groupId>org.jacoco</groupId>
70   <artifactId>jacoco-maven-plugin</artifactId>
71   <version>0.8.12</version>
72 </dependency>
73
74 <dependency>
75   <groupId>org.springframework.boot</groupId>
76   <artifactId>spring-boot-starter-security-test</artifactId>
77   <scope>test</scope>
78 </dependency>
79 <dependency>
80   <groupId>org.springframework.boot</groupId>
81   <artifactId>spring-boot-starter-test</artifactId>
82   <scope>test</scope>
83 </dependency>
84 <dependency>
85   <groupId>org.springframework.boot</groupId>
86   <artifactId>spring-boot-starter-thymeleaf-test</artifactId>
87   <scope>test</scope>
88 </dependency>
89 <dependency>
90   <groupId>org.springframework.boot</groupId>
91   <artifactId>spring-boot-starter-webmvc-test</artifactId>
92   <scope>test</scope>
93 </dependency>
94 </dependencies>
```

```
118 <!-- Paso 2: JaCoCo como plugin -->
119 <plugin>
120   <groupId>org.jacoco</groupId>
121   <artifactId>jacoco-maven-plugin</artifactId>
122   <version>0.8.12</version>
123   <executions>
124     <execution>
125       <id>default-prepare-agent</id>
126       <goals>
127         <goal>prepare-agent</goal>
128       </goals>
129     </execution>
130     <execution>
131       <id>default-report</id>
132       <goals>
133         <goal>report</goal>
134       </goals>
135     </execution>
136     <execution>
137       <id>default-check</id>
138       <goals>
139         <goal>check</goal>
140       </goals>
141       <configuration>
142         <rules>
143           <rule>
144             <element>BUNDLE</element>
145             <limits>
146               <limit>
147                 <counter>COMPLEXITY</counter>
148                 <value>COVEREDRATIO</value>
149                 <minimum>0.60</minimum>
150               </limit>
151             </limits>
152           </rule>
153         </rules>
154       </configuration>
155     </execution>
156   </executions>
157 </plugin>
158 </plugins>
159 </build>
160 </project>
```


Limpieza de compilaciones antiguas, ejecución de pruebas unitarias y reporte:

```
C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2203-2026-201-main>mvnw clean instal
[INFO] Scanning for projects...
[INFO] -----< com.duoc:seguridadcalidad >-----
[INFO] Building seguridadcalidad 0.0.1-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.5.0:clean (default-clean) @ seguridadcalidad ---
[INFO] Deleting C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2203-2026-201-main\t
arget
[INFO] --- jacoco:0.8.12:prepare-agent (default-prepare-agent) @ seguridadcalidad ---
[INFO] argLine set to "-javaagent:C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2203-2026-201-main\target\j
\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2203-2026-201-main\target\j

[INFO] Tests run: 66, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.4.2:jar (default-jar) @ seguridadcalidad ---
[INFO] Building jar: C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2203-2026-201-m
ain\target\seguridadcalidad-0.0.1-SNAPSHOT.jar
[INFO] --- spring-boot:4.0.3:repackage (repackage) @ seguridadcalidad ---
[INFO] Replacing main artifact C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2203-
2026-201-main\target\seguridadcalidad-0.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201
-main (2)\cdy2203-2026-201-main\target\seguridadcalidad-0.0.1-SNAPSHOT.jar.original
[INFO] --- jacoco:0.8.12:report (default-report) @ seguridadcalidad ---
[INFO] Loading execution data file C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2
203-2026-201-main\target\jacoco.exec
[INFO] Analyzed bundle 'seguridadcalidad' with 27 classes
[INFO] --- jacoco:0.8.12:check (default-check) @ seguridadcalidad ---
[INFO] Loading execution data file C:\Users\esteb\OneDrive\Desktop\DUOC\bimestre 6\Seguridad y calidad en desarrollo\semana 8\cdy2203-2026-201-main (2)\cdy2
203-2026-201-main\target\jacoco.exec
[INFO] Analyzed bundle 'seguridadcalidad' with 27 classes
[WARNING] Rule violated for bundle seguridadcalidad: complexity covered ratio is 0.39, but expected minimum is 0.60
[INFO] -----
[INFO] BUILD FAILURE
[INFO] -----
[INFO] Total time: 24.961 s
[INFO] Finished at: 2026-05-03T19:02:24-04:00
[INFO] -----
[ERROR] Failed to execute goal org.jacoco:jacoco-maven-plugin:0.8.12:check (default-check) on project seguridadcalidad: Coverage checks have not been met. S
ee log for details. -> [Help 1]
[ERROR]
[ERROR] To see the full stack trace of the errors, re-run Maven with the -e switch.
[ERROR] Re-run Maven using the -X switch to enable full debug logging.
[ERROR]
[ERROR] For more information about the errors and possible solutions, please read the following articles:
[ERROR] [Help 1] http://cwiki.apache.org/confluence/display/MAVEN/MojoExecutionException
```

El reporte de jaCoCo de la capa Frontend

seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.duoc.seguridadcalidad		57 %		36 %	124	205	278	572	99	169	4	27
Total	943 of 2.224	57 %	46 of 72	36 %	124	205	278	572	99	169	4	27

Cobertura de complejidad en aproximadamente un **39.5%** en la capa Frontend

com.duoc.seguridadcalidad

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
PetRestController		6 %		0 %	12	14	30	35	6	8	0	1
Invoice		0 %		n/a	20	20	41	41	20	20	1	1
InvoiceController		41 %		50 %	4	10	28	46	1	6	0	1
AuthController		13 %		0 %	4	6	20	25	3	5	0	1
PatientRestController		14 %		0 %	6	8	16	21	2	4	0	1
AppointmentRestController		14 %		0 %	6	8	16	21	2	4	0	1
JwtCookieService		32 %		28 %	8	11	18	29	2	4	0	1
Appointment		0 %		n/a	14	14	28	28	14	14	1	1
InvoiceLineItem		0 %		n/a	14	14	28	28	14	14	1	1
Patient		42 %		n/a	11	14	17	28	11	14	0	1
InvoiceCreateRequest		0 %		n/a	9	9	14	14	9	9	1	1
AppointmentRepository		40 %		0 %	3	4	5	8	2	3	0	1
BackendService		98 %		100 %	0	25	3	167	0	17	0	1
AuthRequest		64 %		n/a	2	5	2	7	2	5	0	1
PatientController		33 %		n/a	3	4	3	4	3	4	0	1
InvoicePageController		33 %		n/a	3	4	3	4	3	4	0	1
AppointmentController		33 %		n/a	3	4	3	4	3	4	0	1
SeguridadcalidadApplication		37 %		n/a	1	2	2	3	1	2	0	1
LoginController		60 %		n/a	1	2	1	2	1	2	0	1
WebSecurityConfig		100 %		n/a	0	11	0	29	0	11	0	1
PatientRepository		100 %		100 %	0	4	0	8	0	3	0	1
InvoiceLineItemType		100 %		n/a	0	1	0	4	0	1	0	1
HomeController		100 %		n/a	0	3	0	5	0	3	0	1
AuthResponse		100 %		n/a	0	2	0	4	0	2	0	1
JwtAuthenticationFilter		100 %		n/a	0	2	0	3	0	2	0	1
PetController		100 %		n/a	0	3	0	3	0	3	0	1
JwtUtil		100 %		n/a	0	1	0	1	0	1	0	1
Total	943 of 2.224	57 %	46 of 72	36 %	124	205	278	572	99	169	4	27

Para aumentar los porcentaje de cobertura integro las clases ModelTest y PatientRestControllerTest

```
1 package com.duoc.seguridadcalidad;
2
3 import org.junit.jupiter.api.Test;
4 import java.math.BigDecimal; // Importante agregar esto
5 import static org.junit.jupiter.api.Assertions.*;
6
7 class ModelTests {
8
9     @Test
10     void testInvoiceModel() {
11         Invoice invoice = new Invoice();
12         invoice.setId(id: 1L);
13
14         // CORRECCIÓN: Usar BigDecimal.valueOf para coincidir con el modelo
15         BigDecimal montoPrueba = BigDecimal.valueOf(100.0);
16         invoice.setTotal(montoPrueba);
17
18         assertEquals(1L, invoice.getId());
19         // Comparamos BigDecimal con BigDecimal
20         assertEquals(montoPrueba, invoice.getTotal());
21     }
22
23     @Test
24     void testAppointmentModel() {
25         Appointment app = new Appointment();
26         app.setId(id: 1L);
27         app.setReason(reason: "Consulta");
28
29         assertEquals(1L, app.getId());
30         assertEquals("Consulta", app.getReason());
31     }
32 }
```

```
pom.xml 1 InvoiceController.java ModelTests.java PatientRestControllerTests.java InvoiceControllerTests.java
cdy2203-2026-201-main > src > test > java > com > duoc > seguridadcalidad > PatientRestControllerTests.java > PatientRestControllerTests
1 package com.duoc.seguridadcalidad;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.beans.factory.annotation.Autowired;
5 import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
6 import org.springframework.boot.test.context.SpringBootTest;
7 import org.springframework.test.context.bean.override.mockito.MockitoBean; // Paquete para Spring Boot 3.4
8 import org.springframework.test.web.servlet.MockMvc;
9
10 import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
11 import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;
12
13 @SpringBootTest
14 @AutoConfigureMockMvc(addFilters = false)
15 class PatientRestControllerTests {
16
17     @Autowired
18     private MockMvc mockMvc;
19
20     @MockitoBean // Se usa MockitoBean en lugar de MockBean en la 3.4.0
21     private PatientRepository patientRepository;
22
23     @Test
24     void testGetAllPatients() throws Exception {
25         mockMvc.perform(get("/api/patients"))
26             .andExpect(status().isOk());
27     }
28 }
```



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.